02-Estructuras de Datos Lineales: Listas, Pilas y Colas

Diplomado Data Engineer

Universidad de Santiago de Chile

UNIVERSIDAD DE SANTIAGO DE CHILE

Juan Pablo Castillo

Estructuras de Datos Lineales

Muchas aplicaciones necesitan manipular datos siguiendo algún orden en particular, en lugar de necesitar buscar sobre ellos.

<u>Por ejemplo:</u>

- **Orden cronológico en que ocurren eventos**
  - Ordenes de compra que llegan a un sistema
  - Procesos de impresión que llegan a una impresora
- **Procesar datos en orden cronológico reverso**
  - Operación “deshacer” (ctrl+z) en editores de texto
  - Función volver en navegadores web

Estructuras de Datos Lineales

- A continuación veremos las estructuras de datos lineales más importantes:

  1. Listas

  2. Pilas

  3. Colas

- La característica principal de las estructuras de datos lineales es que: cada **elemento tiene un único predecesor y un único sucesor**.

  - Exceptuando el primer y último elemento.

- Eso si, antes veremos una estructura de datos primitiva que nos permitirá entender bastantes cosas a futuro.

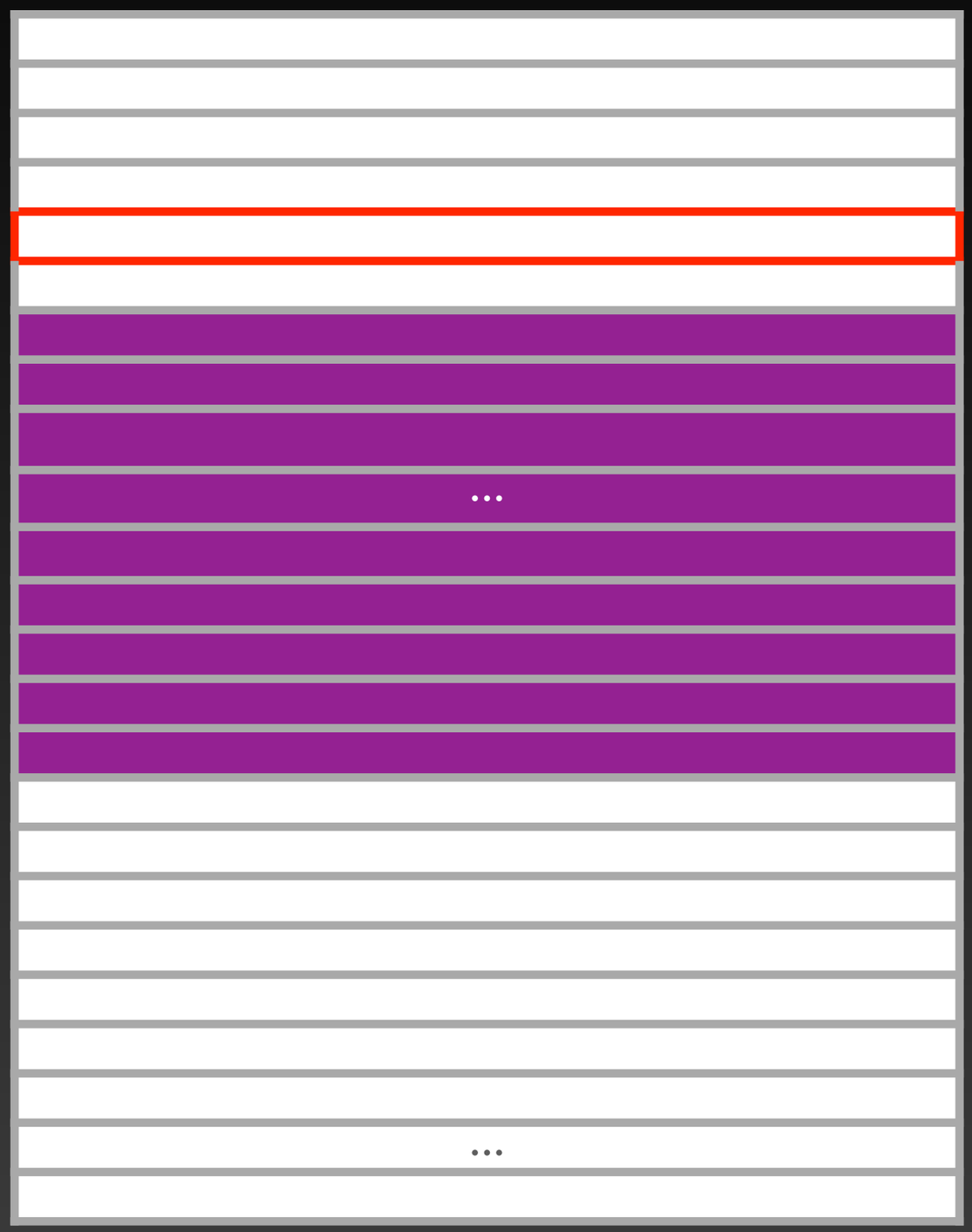
Arreglos

Arreglos

- Los arreglos son quizás la estructura de datos más básica que estudiaremos.
  - Son provistos como un tipo de datos por la mayoría de los lenguajes de programación
- Permiten almacenar un conjunto de datos en memoria principal.
- ***Los elementos del arreglo son almacenados de manera contigua dentro de la memoria***
- Los elementos del arreglo se acceden usando un índice: primer elemento, segundo elemento, etc.

Arreglos

Memory RAM



Conjunto de celdas donde se pueden almacenar datos

Celda donde se puede almacenar un dato

Arreglo, conjunto de celdas contiguas en la RAM

Una característica importante es que ***son estáticos***, no pueden variar su tamaño a lo largo de la ejecución del programa

Listas

Listas

Una lista es una secuencia finita y **ordenada** de datos, donde:

- Cada dato almacenado es llamado “elemento”
- Se dicen que son ***“ordenadas”***, en el sentido que cada elemento tiene asociado una posición dentro de la lista:
  - *Primer elemento, segundo elemento, ..., último elemento*
- Dependiendo la implementación los datos pueden ser del mismo tipo o distintos
  - En *python* por ejemplo pueden ser distintos
- Podríamos resumir su definición como:

$$L = \langle e_0, e_1, e_2, \ldots, e_{n-1} \rangle$$

- Una característica importante, es que ***son dinámicas***, pueden crecer y decrecer a lo largo de nuestro programa.

Listas

Listas en Python

- Las listas en Python están implementadas usando la estructura de datos más primitiva: arreglos.

- Las listas en python se representan con los símbolos: “`[]`” o con “`list()`”

  - Por ejemplo una lista vacía, se podría declarar de las siguientes dos formas:

```
1.L = []
2.L = list()
```

- También podemos definirlas con valores iniciales

```
1.L = [1, 2, 3, 10, 0, 1.0, ‘a’]
2.L = list((1, 2, 3, 10, 0, 1.0, ‘a’))
```

  - Note que los elementos pueden ser de distinto tipo

Operaciones no Mutadoras

- `len()`: retorna el largo de la lista (i.e., la cantidad de elementos).
  - Por ejemplo, `len(L)`
- `L[i]`: obtenemos el *i*-ésimo elemento de la lista
  - Las listas están indexadas desde 0 hasta $n-1$.
- `value in Lista:` podemos preguntar si un elemento está presente en una lista.
  - Retorna `True` si `value` está en la lista y `False` en otro caso.
  - `5 in L`, estamos preguntando si el 5 está en la lista L.

Operaciones de las Listas en Python

- `L.count(x)`: retorna la cantidad de veces que aparece `x` en la lista `L`.
  - Por ejemplo, `L.count(10)`, cuenta la cantidad de veces que aparece `10` en L
- `L.index(x)`: retorna el índice de la primera aparición de `x` en `L`.
  - Por ejemplo,

    ```
    L = [1, 2, 3, 10, 0, 1.0, 10]
    L.index(10)
    ```

    - Retorna el indice `3` donde aparece por primera vez el elemento `10` en `L`
- `Lista1==Lista2`: podemos comparar listas, en este caso compara si son exactamente la misma lista (incluso en el orden de los elementos).
  - Retorna `True` si las listas son ***idénticas*** y `False` en otro caso.
    - `[1, 2, 3, 4, 5] == [1, 2, 3, 4, 5]` —> retorna `True`
    - `[1, 2, 3, 4, 5] == [1, 5, 4, 3, 2]` —> retorna `False`, aunque contengan los mismos elementos, también deben coincidir en el orden.

Operaciones de las Listas en Python

- `L[j:k]:` retorna la sub-lista desde de los elementos entre los indices `j` y `k-1`.
  - Por ejemplo,

    ```
    L = [1, 2, 3, 10, 0, 1.0, 10]
    print(L[2:5])
    ```

    - Retorna la sublista: `[3, 10, 0,]`

- `L1 + L2:` Se pueden concatenar dos o más listas, es decir, se puede crear una nueva lista con los elementos de `L1` y `L2` (en ese orden).
  - Por ejemplo,

    ```
    L1 = [1, 2, 3]
    L2 = [4, 5, 6, 7, 8]
    ```

  - El resultado de `L1 + L2` es una nueva lista con `[1, 2, 3, 4, 5, 6, 7, 8]`

- `c*L:` podemos repetir el contenido de una lista `c` veces para crear una lista más grande.
  - Por ejemplo,

    `2*[1, 2, 3]` da como resultado la lista `[1, 2, 3, 1, 2, 3]`

Tiempo de Ejecución de Operadores

Operación	Tiempo	Descripción
`len(L)`	*O(1)*	-
`L[j]`	*O(1)*	-
`L.count(value)`	*O(n)*	*Debe recorrer toda la lista.*
`L.index(value)`	*O(k+1)*	*Cantidad de elementos comparados hasta el indice* k
`value in L`	*O(n)*	*En el peor caso hay que buscar en toda la lista.*
`L1 == L2 (También: !=, <, <=, >, >=)`	*O(k+1)*	*Cantidad de elementos comparados hasta el indice* k
`L[j:k]`	*O(k-j)*	*Tiempo proporcional a la cantidad de elementos entre los indices* j *y* k
`L1 + L2`	*O(n1+n2)*	*Tiempo proporcional al tamaño de ambas listas.*
`c * L`	*O(cn)*	*Hay que replicar la lista* L, c *veces en una nueva lista*

Operadores Mutadores

- `L[i] = value`: podemos acceder a una casilla y asignarle algún valor.
  - Por ejemplo,

```
L = [1, 2, 3, 4, 5]
L[2] = -1
```

  - Ahora la lista `L` sería [1, 2, -1, 4, 5]
- `L.append(value)`: podemos agregar elementos al final de la lista.
  - Por ejemplo,

```
L = [1, 2, 3, 4, 5]
L.append(6)
```

  - Agrega el elemento 6 al final de la lista, ahora la lista quedaría de la siguiente forma:

```
[1, 2, 3, 4, 5]
```

- `L.insert(i, value)`: agregamos `value` en la posición `i`
  - Por ejemplo,

```
L = [1, 2, 4, 5]
L.insert(2, 3)
```

  - La lista `L` quedaría de la siguiente forma:

```
[1, 2, 3, 4, 5]
```

Operadores Mutadores

- `L.pop()`: eliminamos el último elemento de L.
  - Por ejemplo,

```
L = [1, 2, 3, 4, 5]
L.pop()
```

    - Ahora la a la lista L le falta el último elemento:

```
L = [1, 2, 3, 4]
```

  - También podemos eliminar un elemento en alguna posición arbitraria `i`, `L.pop(i)` ó `del L[i]`.
- `L.remove(value)`: eliminamos la primera ocurrencia de `value` en `L`.
  - Por ejemplo,

```
L = [1, 2, 4 , 3, 4]
L.remove(4)
```

  - Ahora la lista `L` está sin la primera ocurrencia del elemento con valor 4:

```
[1, 2, 3, 4]
```

Operadores Mutadores

- `L1.extend(L2):` se extiende la lista `L1` con los elementos de `L2`.
  - Por ejemplo,

```
L1 = [1, 2, 3, 4, 5]
L1.extend([6, 7, 8])
print(L1)
```

    - El resultado es la que lista `L1` ahora es:

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

  - El comportamiento es idéntico con `L1 += L2`

- `L.reverse():` invierte la lista.
  - Por ejemplo,

```
L = [1, 2, 3, 4, 5]
L.reverse()
```

    - El resultado es la lista `[5, 4, 3, 2, 1]`

Tiempo de Ejecución de Operadores

Operación	Tiempo de	Descripción
`L[i] = val`	*O(1)*	-
`L.append(value)`	*O(1)*	-
`L.insert(i, value)`	*O(n-i+1)*	*Debe desplazar los n-i valores de la lista a la derecha.*
`L.pop()`	*O(1)*	*Solo debe borrar el último elemento*
`L.pop(i)` ó `del L[i]`	*O(n-i)*	*Debe desplazar los n-i valores a izquierda de la lista.*
`L.remove(value)`	*O(n)*	*En el peor de los casos debe recorrer toda la lista*
`L1.extend(L2)` ó `L1 += L2`	$O(n_2)$	*Agrega los* n_2 *elementos de L2 en L1*
`L.reverse()`	*O(n)*	*Debe recorrer toda la lista para invertiral*
`L.sort()`	*O(n log n)*	*Usa una variante de QuickSort (solo como dato)*

Listas Anidadas

- Lo último que veremos de las listas, es que podemos tener listas de lo que sea, incluso de listas.
- A esto lo llamaremos listas anidadas.
- Por ejemplo podríamos tener la siguiente lista:

```
L = [[1, 2, 3, 4], ['a', 'b', 'c', 'd'], [9, 10, 11, 12]]
```

- Y podemos acceder a los cada datos de la siguiente forma:
  - `L[i]`: a la `i`-ésima lista
  - `L[i][j]`: al `j`-ésimo elemento de la `i`-ésima lista.

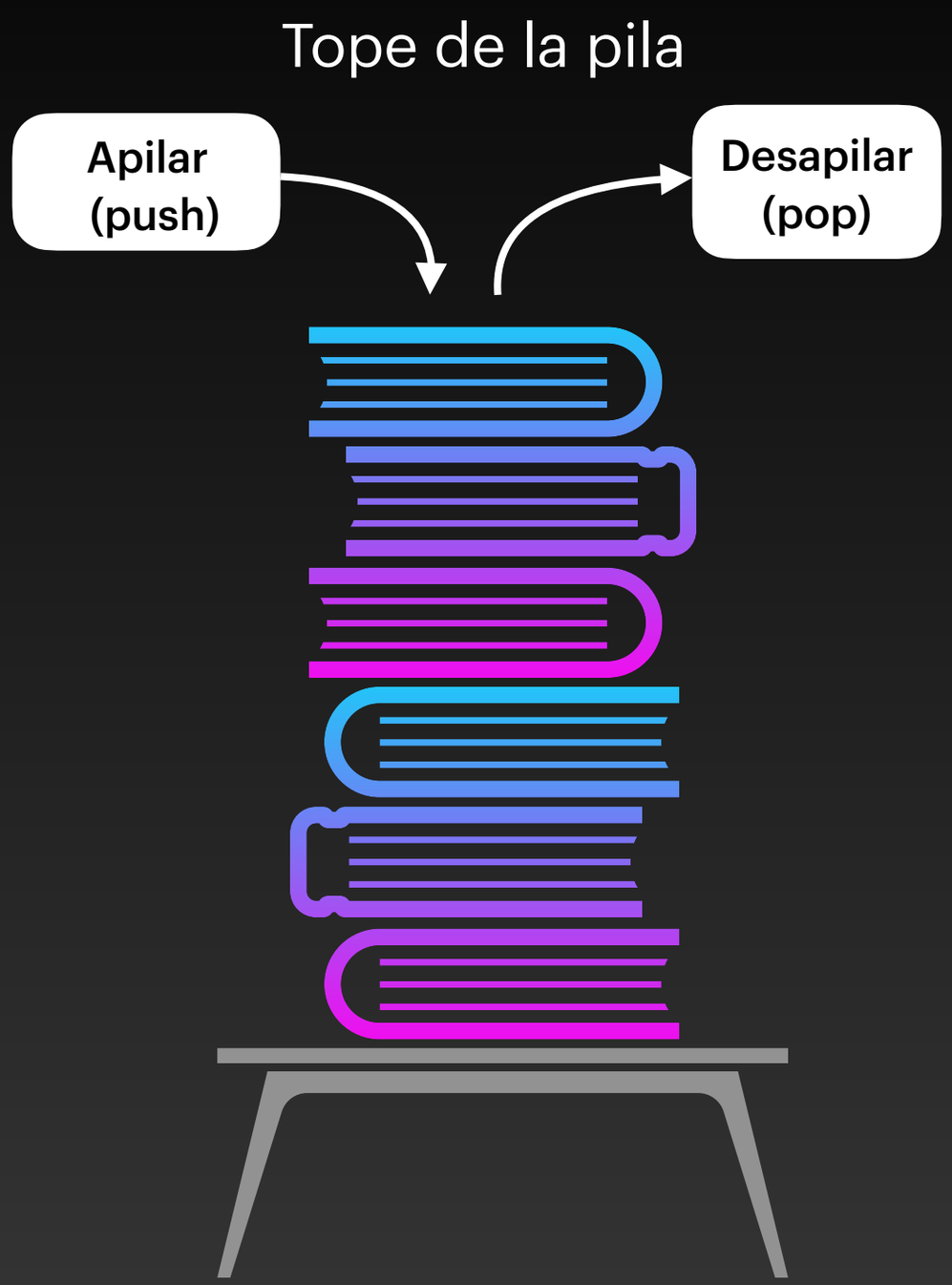
Veamos un caso práctico!

Pilas

Pilas

- Las pilas (stacks) son una de las estructuras más simples pero a la vez bastante usadas en informática.
- Representan el concepto de apilar objetos, por ejem
  - Una pila de sillas,
  - Una pila de platos,
  - Una pila de libros,
  - Etc.
- Es una estructura lineal que permite trabajar **solo** con el **elemento que está al tope de la lista.**

Pilas



- Las pilas son similares a las listas, en la cual solo se puede insertar y borrar elementos en unos de sus extremos.
  - Siendo específicos, solo en el tope
- Se puede representar como una lista que crece y decrece solo por el extremo “superior”
- El **último elemento** que se añade, es el único que se puede consultar y eliminar.

Pilas

Las pilas siguen una política de inserción/borrados conocida como **LIFO.**

- **<u>Last In - First Out</u>:** último en entrar en la estructuralistas, es el primero en salir de ella.

- Esto quiere decir que los elementos son borrados en el orden inverso de su inserción.
- <u>Aplicaciones:</u>
  - Historial de navegación en web-browsers
  - Operación *"undo"* en editores de texto (ctrl+z)
  - Implementación de recursividad
  - Se utiliza como estructura auxiliar en algoritmos más complejos

Pilas

Cómo las utilizamos en Python?

- Ya que solo queremos acceder, insertar y borrar al tope de la pila, podemos simular una pila utilizando las listas.
- Podemos usar `append(value)` para agregar elementos al tope.
- Y el operador `pop()` para eliminar elementos al final de la lista.
  - Recuerde que cuando no pasamos la posición como parámetro elimina el último elemento.
  - Es importante este punto, ya que eliminar el último elemento nos toma tiempo $O(1)$.
- Esto nos garantiza que acceder, insertar y borrar nos cuesten tiempo $O(1)$.

Colas

Colas

- Las colas (*queues*) son otra estructura muy utilizada en informática.
- Son muy utiles ya que se asemejan bastante a paralelismos de la vida real:
  - Colas de espera en bancos, supermercados, etc.
- Esto hace que sean de suma importancia en simulaciones
- Hay muchos casos en donde aparecen:
  - Procesos que esperan a ser ejecutados en sistemas operativos
  - Acceso a recursos compartidos (colas de impresión)
    - Colas de espera en eventos masivos online (cyber, conciertos, etc)

Colas

- Las colas son similares a las listas:
  - Solo se puede insertar elementos en uno de sus extremos, y
  - Se borran el extremo contrario
- El primer elemento de la cola es el único que se puede consultar y eliminar.
- Permite memorizar el orden exacto de una secuencia de eventos.

Colas

Las colas siguen una política de inserción/borrados conocida como **FIFO**.

- **First In - First Out**: primero en entrar a la estructuralistas, es el primero en salir de ello.

- Esto implica que los elementos son borrados en el mismo orden de su inserción
- <u>Aplicaciones</u>:
  - Todo tipo de aplicaciones en donde se necesite implementar algún tipo de cola de espera.

Colas

Cómo las utilizamos en Python?

- A diferencia de las pilas, es difícil simular de forma eficiente su comportamiento usando las listas de Python.
- Por que?
  - Para borrar el primer elemento tendríamos que utilizar `pop(0)`
  - Este tiene un costo $O(n)$, es decir, un costo proporcional a la cantidad de elementos que almacena la cola.
  - Por lo tanto, cada vez que realicemos un `dequeue` nos tocará pagar $O(n)$.
    - Imagine tener que atender `n` clientes, y cada vez que atiende a uno, tiene que avisarle a cada uno que tiene que avanzar.
      - Lento!!!
  - Para esto es mejor utilizar `queue` del modulo `Queue` de Python.

Actividad

- En uvirtual hay un buzón con nombre: “Actividad 2”
  - En este podrá encontrar un Jupyter notebook con el enunciado de la actividad.
  - Desarrolle su actividad en el mismo Jupyter notebook.
- Plazo de entrega: Viernes 17 de abril 18:59hrs.